EPSI
L'école d'ingénierie informatique

PROJECT RESILIENCE ALPHA

Dossier d'ingénierie et de gouvernance

Workshop 2026 — BAC+5

SafeFlow — Groupe G2

Membres du groupe

- Sofiane CHELGHOUM
- Yanis KREIR
- Walid AIT DAOUD
- Gabriel ROBERT
- Hafsa BELKASMI
- Hafsa KODAD

Sommaire

1. Résumé exécutif — p. 2
2. Besoin métier et périmètre — p. 3
3. Architecture — p. 4
4. Composants et décisions techniques — p. 5
5. Scénario d'incident — p. 6
6. Détection ML — p. 7
7. Remédiation et preuve de reprise — p. 8
8. Observabilité — p. 9
9. Sécurité et gouvernance — p. 10
10. Résultats des essais — p. 11 et 12
11. PCA/PRA, limites et perspectives — p. 13
12. Contribution des six membres — p. 13
13. Annexes A à E — p. 14 à 17

1. Résumé exécutif

Objectif

Démontrer qu'un service de paiement fictif, rendu indisponible par l'abus d'une fonction légitime, peut être rétabli **sans intervention humaine**, et que ce rétablissement est **prouvé par une transaction métier** plutôt que par l'état d'un conteneur.

Scénario testé

Un client authentifié de faible privilège (`loadbot`) enchaîne, depuis un second PC, des demandes de devis valides sur `POST /api/quotes`. Chaque devis occupe 0,6 s un des **quatre** emplacements de capacité aval simulée, partagés avec les paiements : la capacité sature et les paiements de l'opérateur sur `POST /api/payments` reçoivent `503`. Les requêtes étant valides, le WAF ne les bloque pas ; la détection repose donc sur le *comportement* (métriques et journaux), pas sur une signature. La charge est bornée à 20 req/s, 12 requêtes simultanées, 700 requêtes et 35 s au maximum. Ce n'est pas une attaque distribuée : c'est un générateur de charge mono-poste, mono-identité.

Résultat principal

22 reprises complètes sont attestées par les preuves d'exécution (`runtime/evidence/acceptance-*.json`) : incident `recovered`, transaction témoin validée, total du registre exactement à 200 000 centimes et quatre étapes horodatées présentes. Le délai détection → service rétabli vaut **2,47 s au minimum, 2,83 s en médiane, 6,88 s au maximum** — toutes sous la cible interne de 60 s.

Le cycle du **22 octobre 09:05 UTC** est le premier dont les deux côtés sont horodatés : le récapitulatif du PC Attaquant donne le début exact de la charge, ce qui permet enfin de mesurer la chaîne complète plutôt que de l'estimer. **Charge → détection : ~5,5 s. Charge → transaction témoin validée : ~12,3 s.** Les trois fichiers de preuve de ce cycle se recoupent à la seconde près (§5.4, §10.1).

Un **mode témoin sans remédiation** a été ajouté pour mesurer l'écart. Dans ce mode, le détecteur continue de mesurer, d'alerter et de journaliser, mais le remédiateur n'applique aucun confinement pendant une fenêtre bornée. Les deux essais témoins exploitables montrent **48 et 54 paiements fictifs refusés** sur ~121 sondes, contre **12 à 16 sur ~115** en mode protégé : l'admission ciblée divise par environ quatre la proportion de paiements refusés sous la même charge.

Limites, dites franchement

- La reprise démontrée est un **retour de disponibilité**. Elle ne corrige pas une compromission, ne supprime pas l'attaquant et ne couvre pas une attaque répartie sur plusieurs identités.
- Le **rappel de détection n'est pas mesuré** : `ml-report.json` porte explicitement `attack_recall: "NOT MEASURED"`. Sur 39 observations d'essai, 14 n'ont vu aucun incident — attaque non lancée ou charge insuffisante.
- Le projet **n'est pas conforme** à NIS2 ou DORA et ne le prétend pas : ces textes ont orienté des décisions, les écarts sont listés au §9.6.
- Point de défaillance unique assumé : un seul PC Hôte, une seule base, pas de TLS applicatif.

Traçabilité : la preuve de référence enregistre c11ec6f et dirty: true ; le dépôt est actuellement à 4a1430d.
Le lien exact entre le code exécuté et ce commit n'est pas établi ; les résultats restent rattachés aux preuves citées.

2. Besoin métier et périmètre

2.1 Hypothèse d'activité

L'entreprise fictive *ALPHA PAY* traite des paiements pour un marchand fictif. Sa fonction prioritaire est le **traitement fiable et traçable d'une transaction**. Une interruption retarde les demandes, dégrade la confiance et crée une charge d'exploitation. Aucun chiffre financier réel n'est fourni : l'hypothèse illustrative retenue en atelier de gouvernance (10 transactions/min × marge fictive de 0,10 €, soit 1 €/min de contribution différée) sert à hiérarchiser, pas à chiffrer une perte.

2.2 Ce que « paiement » désigne ici

Le registre tient deux comptes, `alice` et `merchant`, initialisés à 100 000 centimes chacun. Les montants sont des **entiers en centimes**, de 1 à 100 000 ; il n'y a ni carte, ni IBAN, ni appel à un prestataire externe. Débit, crédit et écriture de transaction sont validés dans une seule transaction base de données, les lignes étant verrouillées dans un ordre déterministe. **Le total doit rester exactement 200 000 centimes** : c'est l'invariant que la preuve de reprise contrôle. Une clé d'idempotence identique avec le même contenu renvoie la transaction existante ; la même clé avec un contenu différent renvoie `409`.

2.3 Exigence de continuité retenue

Objectif	Définition retenue	Statut
RTO (*Recovery Time Objective*)	45 s entre la première indisponibilité observée et une transaction confirmée.	cible, voir §11
RPO sur ce scénario	Aucune transaction déjà confirmée perdue. Le confinement ne réinitialise ni compte ni donnée.	tenu : total à 200 000
RPO en cas de perte Hôte/base	Dépend de la dernière sauvegarde réellement disponible.	non résolu
Cible interne du chrono	Détection → transaction témoin validée en moins de 60 s (45 s objectif resserré).	tenu sur 22/22

RTO et chrono ne mesurent pas la même chose : le chrono part de la *détection* (heure d'émission de l'alerte par le détecteur), le RTO part de la *première indisponibilité constatée*. Les deux sont rapportés séparément au §11.

2.4 Périmètre d'essai et interdits

Dans le périmètre

- PC Hôte Windows 10/11, Docker Desktop + WSL2, l'ensemble de la pile.
- PC Attaquant : uniquement le client de charge Python, connecté au seul point d'accès du groupe.
- Cible unique : `http://192.168.137.1:8081`, le WAF, seul point d'entrée publié.
- PC IA optionnel : Ollama seul.

Hors périmètre, explicitement

- Réseau du campus, autre groupe, équipement tiers, Internet.
- Scan réseau, exploit, élévation de privilèges, persistance, exfiltration.
- Toute cible qui n'est pas une IP littérale du sous-réseau du lab : refusée *avant* connexion par `resolve_target`.

> **Garde-fou vérifié par test.** Le client de charge exige l'option `--lab-authorized` et refuse de démarrer sans elle ; il refuse un nom d'hôte, une adresse du campus ou d'Internet, et ne suit aucune redirection. Ces refus sont couverts par `tests/test_configs.py` (`test_attack_target_stays_inside_lab`, `test_attack_refuses_without_ack_or_outside_lab`), qui vérifient aussi que la demande de mot de passe n'est jamais atteinte quand le périmètre est violé.

2.5 Ce que le prototype ne prétend pas être

Ni une plateforme de paiement, ni un SIEM, ni une solution de haute disponibilité. La faiblesse exploitée — un devis lent partageant la capacité des paiements — est un **défaut de conception volontaire et documenté**, introduit pour rendre le service réellement saturable sans attaquer l'OS, la mémoire ni le réseau. Une architecture industrielle séparerait les pools devis et paiement, avec quotas, files et budgets de ressources dès la conception.

3. Architecture

Quinze conteneurs, huit réseaux Docker dont quatre strictement internes, un seul port publié. Le schéma distingue le *flux de requêtes* (traits pleins) de la *boucle de détection et de remédiation* (traits tiretés).

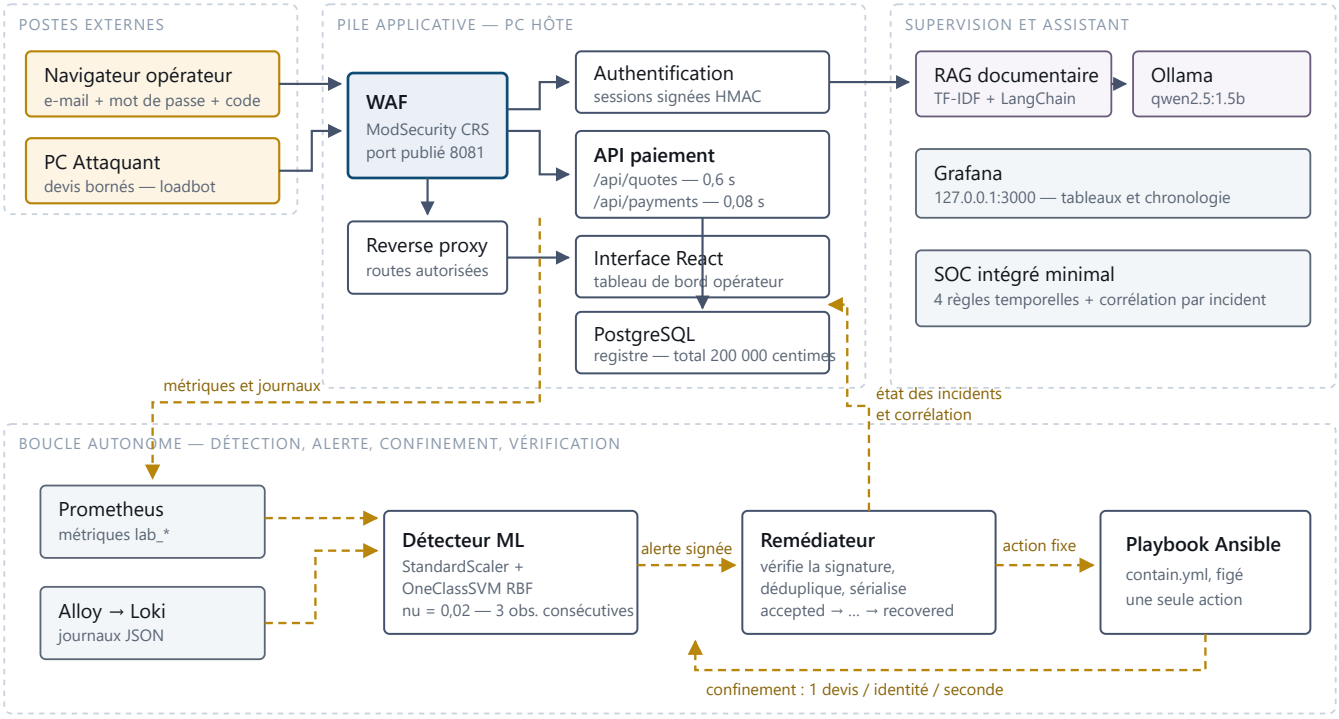


Figure 1 — Flux de requêtes (traits pleins) et boucle autonome détection → alerte → confinement → vérification (traits tiretés). Aucun socket Docker n'est confié à un service ; le RAG ne reçoit aucun secret de contrôle et n'exécute aucun outil. Source : `compose.yaml`, `infra/nginx/default.conf`, `docs/02-ARCHITECTURE.md`.

3.1 Exposition réseau

Seul le WAF est publié, sur `${LAB_BIND}:${LAB_PORT}` (8081 par défaut : 8080 est déjà pris sur le poste Hôte). Grafana reste sur `127.0.0.1:3000`. Base, Prometheus, Loki, Alloy, le détecteur, le remédiateur et Ollama **n'exposent aucun port**. Les réseaux `app`, `data`, `control` et `observe` sont déclarés `internal: true`. Les routes externes sont énumérées explicitement dans le reverse proxy ; tout le reste renvoie `404`. Il n'existe aucun chemin public vers `/control/contain` (service de paiement) ni vers `/alerts` (remédiateur).

3.2 Cloisonnement réseau

Huit réseaux Docker, dont **quatre déclarés** `internal: true` : ni accès sortant, ni publication possible. Un service n'est joignable que par ceux qui partagent un de ses réseaux — la base ne l'est que par le paiement, l'action de confinement que par le remédiateur, et les métriques et journaux ne sortent pas de l'Hôte. Le détail réseau par réseau figure en **annexe B**.

3.3 Chaîne visée, en une phrase

Transaction confirmée → charge de devis bornée depuis le second PC → paiements en `503` → détection comportementale sur métriques et journaux → alerte signée HMAC → playbook Ansible fixe activant une admission ciblée → transaction témoin vérifiée *à travers le WAF* → paiement réussi alors que la charge est toujours active.

4. Rôle des composants et décisions techniques

Composant	Version / image	Rôle effectif et point notable
API paiement	FastAPI 0.115.12	`POST /api/quotes` (devis, 0,6 s) et `POST /api/payments` (paiement, 0,08 s) partagent **4 emplacements** de capacité ; au-delà, `503` sans aucune écriture. `POST /control/contain` est interne, protégé par un secret dédié, et persiste la politique sur volume. Un seul worker : le compteur de capacité est en mémoire.
Authentification	FastAPI	Deux chemins. *Opérateur* : e-mail + mot de passe haché + code à 6 chiffres envoyé par courriel (expire en 5 min, 5 essais, 10 demandes/min/adresse). *Comptes de laboratoire* `alice` / `loadbot` : `POST /auth/token` , 30 connexions/min/adresse vue.
Registre	PostgreSQL 17.4-alpine	Comptes et transactions dans une transaction unique, lignes verrouillées en ordre déterministe, montants entiers. SQLite ne sert qu'aux tests natifs et **ne valide pas** PostgreSQL.
WAF	owasp/modsecurity-crs:nginx	Seul point d'entrée publié, filtrage de contenu CRS. **Aucune limitation de débit** n'y est configurée (`infra/nginx/default.conf` n'a ni `limit_req` ni `limit_conn`) : un devis valide passe CRS. Le tag `nginx` est mobile — figer son digest reste à faire.
Détecteur ML	scikit-learn 1.6.1	StandardScaler + OneClassSVM RBF (`gamma='scale'` , `nu=0,02`). Quatre entrées lues toutes les 2 s dans Prometheus et Loki. Journalise chaque observation, ce qui rend la fenêtre d'attaque relisible après coup.
Remédiateur	Ansible Core 2.18.4	Reçoit des alertes signées HMAC-SHA256, valide UUID, fraîcheur (±30 s) et nombres, déduplique durablement, sérialise la correction. Une seule action autorisée. Délai Ansible borné à 25 s. Pas de socket Docker.
Prometheus	v3.2.1	Compteurs `lab_requests_total` , `lab_capacity_denied_total` , jauges `lab_active_work` , `lab_contained` , histogramme `lab_duration_seconds` . Rétention 2 jours.
Alloy → Loki	v1.8.2 → 3.4.2	Expédie les journaux JSON métier et l'audit WAF. Rétention 48 h. Alloy tourne en root *en lecture seule* pour lire les fichiers.
Grafana	11.6.0	Tableaux de bord et panneau de chronologie d'incident. Lié à `127.0.0.1` uniquement.
RAG	LangChain Core 0.3.59	Corpus `docs/*.md` , recherche lexicale TF-IDF (3–5 caractères), 3 extraits au plus, génération par Ollama `qwen2.5:1.5b` à température 0. **Aucun outil d'action, aucun secret de contrôle.** Hors corpus : « Information absente du corpus. » *sans appeler le modèle.*
Interface	React 19 / Vite 6.2.6	Navigation par barre latérale après connexion : vue d'ensemble, service paiement, supervision, services de démonstration. Le chrono est calculé sur les horodatages du serveur, jamais sur une horloge du navigateur.

4.1 Trois décisions structurantes et leur coût

D-1 — On contient, on ne recrée pas le conteneur

Recréer un conteneur exigerait de confier le socket Docker — un pouvoir équivalent à root sur l'Hôte — à un service qui traite des entrées venues du réseau. Le remédiateur reçoit donc deux pouvoirs étroits : appeler une action fixe sur le service de paiement avec un secret dédié, et écrire une liste de bannissement. *Coût :* un conteneur réellement compromis ne serait pas remplacé ; la réponse relève alors d'une décision d'opérateur (confinement, investigation, reconstruction depuis une image saine), hors de l'automate. Le `restart: unless-stopped` de Docker n'est pas une remédiation.

D-2 — OneClassSVM plutôt qu'IsolationForest

Deux des quatre variables — rejets de capacité sur 10 s et leurs journaux — valent exactement 0 sur toute baseline valide, alors que ce sont les signaux directs de l'incident. Un IsolationForest coupe entre le minimum et le maximum appris : une variable constante à l'apprentissage ne peut pas être coupée et n'apporte rien. Avec StandardScaler, scikit-learn n'applique aucune mise à l'échelle à une variable d'écart-type nul : 30 rejets sur 10 s se retrouvent à 30 unités du nuage normal et sortent nettement de la frontière. L'apprentissage du SVM est de plus déterministe, ce qui compte pour une démo reproductible. *Coût :* les deux modèles n'ont pas été comparés sur la baseline réelle ; le choix repose sur l'analyse des données et un test unitaire, pas sur un banc d'essai.

D-3 — Jeton de session signé HMAC, pas un jeton standard

Le service d'authentification émet `base64url(charge).base64url(HMAC-SHA256)` portant identité, rôle, audience `resilience-lab` et expiration à 1 h ; chaque service le vérifie avec la même clé, un secret Docker. Une quarantaine de lignes lisibles et testées, plutôt qu'une bibliothèque de jetons dont il faut maîtriser les options en quatre jours. *Coût, réel :* la clé est symétrique et partagée, donc **tout service qui la détient peut émettre un jeton opérateur**, pas seulement le vérifier — le remédiateur et le RAG la détiennent. Ni rotation, ni révocation avant expiration. En contexte industriel : OIDC et clés asymétriques.

5. Scénario d'incident

5.1 Nature de l'abus

Abus d'une fonction légitime pour saturer une capacité partagée. Le compte `loadbot`, autorisé uniquement à demander des devis, enchaîne des requêtes *valides*. À 20 req/s, les devis réclament une douzaine d'emplacements pour quatre disponibles : la capacité reste saturée et les paiements reçoivent `503`, sans aucune ligne de débit. Ce n'est ni une compromission, ni une fuite de données, ni une élévation de privilèges.

5.2 Bornes appliquées par le code

Le client de charge est borné *par son propre code*, pas par une consigne : 20 requêtes/s au maximum, 12 en vol, 700 au total, une durée de 5 à 35 secondes (30 par défaut), une cible unique `POST /api/quotes`, une IP littérale du sous-réseau du laboratoire, et l'option `--lab-authorized` obligatoire. Le tableau complet, avec le point d'application de chaque borne, est en **annexe B**.

5.3 Symptômes HTTP observés, et ce qu'ils désignent

> **Deux séries de codes HTTP ne doivent pas être confondues.** Les codes de l'**observateur sur l'Hôte** portent sur ses propres **paiements fictifs** (`POST /api/payments`, une sonde toutes les 0,5 s) : ils mesurent l'*impact métier*. Ceux du **récapitulatif du client de charge** portent sur ses **devis** (`POST /api/quotes`) et mesurent l'*effet de l'attaque*, un `429` y étant *attendu* après confinement. Le dossier cite les deux, toujours en nommant leur route. Ce récapitulatif reste sur le PC Attaquant et n'est pas versionné : ses chiffres proviennent de la fiche de l'essai, pas du dépôt. source non versionnée

5.4 Chronologie mesurée — cycle protégé du 22 octobre 2026

Premier cycle dont les **deux côtés** sont horodatés. Les heures viennent du récapitulatif du PC Attaquant (run `92119f71`), des étapes signées de `acceptance-1792659988.json` et de la fenêtre de `m7-proof-20261022T090634Z.json`.

Heure UTC	Δ charge	Événement	Observation et source
09:05:08,46	—	Préparation	Reset de la politique, redémarrage du paiement, `/api/ready` vérifié. recette-run
09:05:15,99	—	Observateur prêt	Transaction initiale `200` : le service est sain avant la charge. acceptance
09:05:23,04	0,00 s	**Charge lancée à la main**	521 requêtes `POST /api/quotes` en 30,4 s depuis le PC Attaquant. récapitulatif attaquant
09:05:24,01	+0,97 s	Premier paiement refusé	La capacité partagée sature : `503` sur `/api/payments`. acceptance
09:05:24,38	+1,35 s	1re observation anormale	Score négatif *et* rejets de capacité présents. M7, `first_anomalous_s`
09:05:28,50	**+5,46 s**	`detected`	3e observation anormale consécutive : alerte signée émise. étape `detected`
09:05:28,57	+5,53 s	`containing`	Alerte acceptée, playbook lancé : **prise en charge en 67 ms**. Concorde avec `first_alert_s` = 48,09 s de la fenêtre M7.
09:05:34,95	+11,91 s	`verifying`	Admission ciblée active et persistée ; healthcheck `/ready` passé.
09:05:35,38	**+12,35 s**	`recovered`	Transaction témoin `695ef145` créée *à travers le WAF*, inscrite au registre, total à 200 000 centimes.
09:05:53,47	+30,44 s	Fin de la charge	Service rétabli alors que la charge tournait encore : **321 des 521 requêtes ont reçu `429`**, ce que seule l'admission ciblée produit.

Le chrono interne (**6,88 s**, détection → témoin) part de l'heure signée `issued_at`, pas de sa réception par le remédiateur. Les deux horloges diffèrent : M7 mesure un écart Hôte − conteneur de **0,8 s**, à retrancher des deltas « Δ charge » comme marge d'incertitude. Le délai de détection ainsi mesuré (~5,5 s) et le délai calibré par rejeu de ~11,7 s (§6.4) ne se comparent pas directement : le rejeu part du début d'une fenêtre d'attaque reconstituée, pas de l'heure réelle de lancement.

5.5 Signaux utilisés pour décider

La décision n'utilise **aucune signature de requête**. Elle combine quatre mesures (débit de devis servis, occupation de la capacité, rejets de capacité, journaux de rejet) et exige *conjointement* un score de nouveauté négatif et la présence de rejets de capacité — une barrière d'autorisation qui évite de corriger une nouveauté inoffensive, sans remplacer le modèle ni servir à juger sa qualité (§6).

6. Détection ML

6.1 Modèle réellement implémenté

Vérifié dans `services/detector/model.py` : un pipeline scikit-learn `make_pipeline(StandardScaler(), OneClassSVM(kernel='rbf', gamma='scale', nu=nu))` — un modèle de **détection de nouveauté à une seule classe**, qui apprend le fonctionnement nominal sans étiquette d'attaque. **Ni classification binaire supervisée, ni IsolationForest, ni modèle préentraîné.** Un score négatif signale une nouveauté.

6.2 Entrées

Variable (dans l'ordre)	Origine réelle	Fenêtre
`quote_rate`	Prometheus — `lab_requests_total{route="quotes"}`	`rate` 10 s
`active_work`	Prometheus — `lab_active_work`	instantané
`capacity_denied_10s`	Prometheus — `lab_capacity_denied_total`	`increase` 10 s
`denial_logs_10s`	Loki — événements `capacity_denied` du paiement	`count_over_time` 10 s

Échantillonnage toutes les 2 secondes ; aucune donnée bancaire n'entre dans le modèle. Une série absente ou une valeur non finie est **refusée** : une panne de collecte n'est jamais remplacée par un fonctionnement normal inventé — le détecteur passe en `telemetry_or_delivery_error` et le dit. Une baseline contenant des rejets de capacité est refusée à l'entraînement.

6.3 Règle d'alerte

Une observation est *anormale* si — et seulement si — le score est négatif **et** que des rejets de capacité sont présents. Une alerte part après **3 observations anormales consécutives**, avec un refroidissement de **30 s** et aucun réentraînement pendant l'incident. Une seule fonction sert au compteur, à `GET /anomaly` et au rejeu de calibration : il n'existe qu'une définition.

6.4 Calibration — et une leçon de méthode

Baseline de **90 observations** sur la pile réelle, 72 pour l'apprentissage et les 18 suivantes en validation chronologique (`ml-report.json`, SHA-256 `0f42ace4…`). Un premier balayage mené *sur ce seul holdout de 18 points* proposait `nu=0,05`, `consecutive=1` à 0 % de fausses séries. **C'était trompeur** : rejoué sur 156 observations normales indépendantes, ce couple produit **9 fausses séries**. L'outil refuse désormais de conclure sous 150 observations, et ce premier fichier porte `conclusive: false`.

Couple	Fausses séries (156 obs. normales)	Scores négatifs par observation	Détecte	Délai
`nu=0,05`, `cons.=1` — proposition trompeuse du 1er passage	9	5,8 %	oui	7,7 s
`nu=0,03`, `consecutive=3` — réglage précédent	2	15,4 %	oui	11,7 s
`nu=0,02`, `consecutive=3` — retenu et déployé	**0**	**14,1 %**	**oui**	**11,7 s**

Preuve : `calibration-20261020T065603Z.json` (`conclusive: true`). Les valeurs déployées sont les défauts du code (`DEFAULT_NU = 0.02`, `DEFAULT_CONSECUTIVE = 3`). Le délai va du début de la fenêtre d'attaque à l'observation qui déclenche l'alerte ; il exclut la livraison et la remédiation.

6.5 Ce que le modèle fait sur une attaque réelle

Sur les **18 preuves de jalon M7 validées** (sur 23 produites), le détecteur a journalisé entre **7 et 19 observations anormales** par fenêtre d'une cinquantaine, score minimal autour de **−0,32**. Le pic est *mesuré*, pas lu sur un écran : débit de devis de **4,88 à 6,63 req/s** contre une enveloppe normale plafonnant à **1,37 req/s**, occupation à **4,0** contre **1,0**, et **81 à 398 rejets de capacité** sur la fenêtre.

Le cycle du 22 octobre (`m7-proof-20261022T090634Z.json`, 4 contrôles sur 4) illustre le cas nominal : **9 observations anormales sur 53**, compteur maximal 6, pic à **6,25 req/s**, **130,4 rejets de capacité**, une alerte livrée. C'est le seul cycle où l'heure de lancement de la charge est connue : le délai mesuré charge → détection y vaut **~5,5 s**, sous la cible de 10 s, à ±0,8 s d'horloge près (§5.4). **Une mesure ne fait pas une performance** : il faut la refaire sur plusieurs cycles.

6.6 Limites du détecteur, sans détour

- **Le rappel n'est pas mesuré.** `ml-report.json` porte `attack_recall: "NOT MEASURED"` : on sait que le modèle reconnaît les observations de la fenêtre d'attaque rejouée, pas quelle fraction d'incidents il détecterait en général.
- **14,1 % des scores sont négatifs sur du trafic normal.** Aucune fausse alerte ne part, car la règle exige en plus des rejets de capacité — nuls en trafic normal — mais ce filet n'est pas une qualité du modèle. Les fenêtres se chevauchant, le holdout de 18 points reste un diagnostic.
- **Non couverts** : attaques lentes, dérive de charge, identités multiples, compromission de la télémétrie. Le volume de baseline reste très limité.

7. Remédiation et preuve de reprise

7.1 Étapes réellement implémentées — mode protégé

État	Ce qui se passe	Garde-fou
`accepted`	Alerte reçue sur `/alerts` (interne, non publiée). Signature HMAC-SHA256 des octets exacts vérifiée ; UUID, fraîcheur ±30 s, score fini négatif, 4 variables finies, rejets > 0.	Corps > 4096 o : `413` ; signature invalide : `403` ; contenu refusé : `422`. Déduplication durable.
`containing`	Le playbook fixe `contain.yml` active l'admission ciblée : **un devis par identité et par seconde**, persistée et idempotente. Le service n'est jamais arrêté.	Playbook, URL de contrôle et action ne viennent **jamais** du corps de l'alerte. Délai borné à 25 s ; un incident à la fois.
`verifying`	Playbook terminé, healthcheck `/ready` passé. Le bannissement optionnel, s'il est actif, est *relu dans le fichier*, pas supposé.	Le ban est au mieux : son échec ne fait jamais échouer le confinement.
`recovered`	**Une disponibilité de base ne suffit pas.** Le remédiateur crée un paiement d'un centime *à travers le WAF*, vérifie son inscription au registre et le total à 200 000 centimes.	Sans ces trois conditions, l'état ne devient pas `recovered`.
`failed`	Toute exception laisse l'incident en échec, avec `manual_review_required: true` et le type d'erreur.	Aucune boucle de commandes ouverte ; aucune fausse réussite.

Un incident interrompu est repris au redémarrage par la *même* action idempotente. Le jeton de service nécessaire à la vérification est obtenu **directement auprès du service d'authentification par le réseau interne**, non à travers le WAF : derrière le proxy, tous les clients publics partagent un budget de connexion qu'un flot pouvait épuiser — la vérification recevait alors `429` et l'incident basculait en `failed`. Corrigé et verrouillé par `tests/test_login_budget.py`, qui épuise le budget public et vérifie que la vérification métier aboutit quand même.

7.2 Mode témoin sans remédiation

Ajouté pour répondre à une question simple : *la charge a-t-elle réellement un effet quand la résolution automatique n'intervient pas ?* Le détecteur continue de mesurer, d'alerter et de journaliser — l'incident est enregistré avec sa vraie heure de détection — mais le remédiateur **n'applique ni confinement, ni bannissement, ni transaction témoin**. L'incident termine dans un état **terminal distinct,** `observed`, impossible à confondre avec `recovered`.

> **Ce n'est pas un interrupteur de sécurité.** Le WAF, la connexion opérateur, la limitation des essais de connexion et toutes les autres défenses restent actifs. Garde-fous implémentés et testés :
>
> - fenêtre **bornée dans le temps** : 5 à 300 s, plafond appliqué par le service ;
> - réactivation automatique **à l'échéance**, vérifiée au point de décision et par la boucle d'entretien, même si personne ne lit l'état ; et par le lanceur dans un `finally` — étape en échec, interruption clavier ou plantage ;
> - **jamais écrite sur disque** : tout redémarrage du remédiateur revient avec la remédiation active ;
> - **opérateur authentifié obligatoire** pour armer comme pour arrêter ; confirmation explicite ; refus si une remédiation est déjà en cours ;
> - état publié à chaque interrogation de `/ops/status` : bandeau d'avertissement **sur toutes les pages** du tableau de bord, avec réactivation immédiate.

Le récapitulatif JSON consigne le mode, la fenêtre, l'impact observé et les contrôles attendus en échec. **Un essai témoin n'est jamais** `PASS` : il porte un statut propre — `CONTROL_IMPACT_OBSERVED`, `CONTROL_NO_IMPACT` ou `CONTROL_NOT_INHIBITED`.

7.3 Pourquoi deux contrôles échouent volontairement en mode témoin

`acceptance.py` et `check_mercredi.py` vérifient une **reprise** — exactement ce que le mode témoin inhibe. Leur `FAIL` est donc attendu, et listé dans `expected_failures` du récapitulatif ; `check_mardi.py`, qui mesure la charge et la détection, doit lui *réussir*. **Les critères** `PASS` **de ces trois scripts n'ont pas été modifiés** : seule la lecture de leur résultat dépend du mode.

7.4 Ce que la preuve de reprise ne couvre pas

- Le bannissement d'adresse est une **surcouche optionnelle** (`compose.ban.yaml`) et **il était désactivé dans tous les cycles mesurés** : la preuve du 22/10 le consigne explicitement (`ban.enabled: false`, *« bans disabled (BAN_ENABLED not set) »*). Les reprises démontrées ici le sont donc **sans aucun bannissement**. Le dépôt prouve la *décision* — filtrage au sous-réseau du lab, exclusion de l'Hôte et des adresses internes, expiration à 5 min, idempotence — mais **pas son application au WAF**, que personne n'a exécutée. à confirmer en exécution
- La vérification ne teste plus la *connexion* à travers le WAF, seulement le paiement.
- Le remédiateur utilise le compte opérateur `alice` et détient déjà la clé de session ; une identité de service dédiée au rôle minimal serait plus propre.

8. Observabilité

8.1 Ce que Prometheus mesure

Cinq séries décrivent le service : le nombre de réponses par route et par code, les rejets de capacité — signal direct de l'incident —, l'occupation des quatre emplacements, un indicateur disant si l'admission ciblée est active, et le temps de traitement par route. Les séries étiquetées sont *pré-touchées* au démarrage : sans cela, après un redémarrage, la requête du détecteur ne renvoie rien et il se déclare en erreur de télémétrie plutôt que de lire un zéro. Noms et types exacts : **annexe B**.

8.2 Ce que le détecteur consigne lui-même

Chaque observation scorée est journalisée comme un événement `inference` portant le score, les quatre variables et le compteur d'alerte. **C'est ce qui rend une fenêtre d'attaque relisible après coup, exactement comme le détecteur l'a vue** : `services.detector.window` l'exporte en jeu de données pour la calibration, ou en résume la fenêtre pour le contrôle de jalon. Les alertes livrées sont journalisées séparément (`alert_delivered`) avec l'identifiant d'incident.

8.3 Grafana et le SOC intégré

Grafana, lié à `127.0.0.1` , présente les séries ci-dessus et un panneau de chronologie d'incident. La fonction SOC est **minimale et assumée comme telle** : quatre règles temporelles sur Loki (au moins 5 échecs de connexion par minute, pression de capacité, livraison d'une alerte ML, activité d'audit WAF) et *une* corrélation qui suit un incident à travers les services par son `incident_id` .

Cette corrélation apporte ce qu'un simple comptage ne voit pas : une alerte **jamais traitée** (silence du remédiateur au-delà de 15 s), une reprise **bloquée** (aucun état final au-delà de 90 s), une reprise déclarée **sans toutes ses étapes** ou dans un ordre incohérent. Son état est journalisé à chaque changement, exposé par `/ops/ml` et affiché sous le chrono du tableau de bord. Ce n'est ni un SIEM, ni un SOC industriel : elle ne suit qu'un seul type d'incident, et l'activité WAF est un signal à examiner, pas la preuve qu'une ligne correspond à une attaque.

8.4 Journaux et fichiers de preuve

Dans Loki (48 h)

- Journaux JSON métier : heure, composant, événement, identifiants fictifs.
- Audit WAF au format AFZ : métadonnées et en-têtes de *réponse*.

Ni corps de connexion, ni jeton, ni clé, ni mot de passe, ni requête SQL contenant un secret. Le remédiateur, lui, ne journalise jamais le bearer qu'il manipule.

Dans `runtime/evidence/`

- `acceptance-<epoch>.json` — observations brutes et bloc `summary` .
- `m7-proof-<UTC>.json` — pic mesuré et verdict du détecteur, *y compris en cas d'échec*.
- `recette-run-<epoch>-<id>.json` — récapitulatif d'un essai, avec son mode.
- `calibration-<UTC>.json` , `ml-report.json` , `ml-baseline.json` .

> **Un échec documenté vaut mieux qu'une absence de trace.** Les contrôles de jalon écrivent leur preuve même quand ils échouent (`passed: false` et la raison). Sur les 22 preuves M7 du dépôt, **5 portent un échec** : quatre « aucun rejet de capacité sur la fenêtre » — l'attaque n'avait pas saturé le service ou la fenêtre était mal placée — dont trois avec « aucune observation anormale ». Ces fichiers sont conservés tels quels.

8.5 Limites de l'observabilité

- Rétention courte : Prometheus 2 jours, Loki 48 h. Une fenêtre plus ancienne n'est plus interrogeable, et le contrôle de jalon le signale explicitement.
- Les journaux métier JSON sur volume **ne tournent pas automatiquement** dans cette version : il faut les archiver ou les purger entre essais maîtrisés.
- La machine Hôte peut altérer ses propres journaux : **aucune preuve d'inaltérabilité** industrielle n'est apportée.
- Les horloges de Windows et des conteneurs diffèrent ; les fenêtres s'expriment en epoch ou en ISO 8601 avec fuseau, et une heure sans fuseau est refusée. Le contrôle de jalon rapporte l'écart d'horloge constaté.

9. Sécurité et gouvernance

9.1 Authentification opérateur et double facteur

La connexion opérateur combine e-mail et mot de passe vérifié contre une empreinte, puis un code à six chiffres généré de façon cryptographique et envoyé à cette adresse. Le serveur conserve une empreinte HMAC du code ; il expire après 5 minutes et le défi est détruit après 5 tentatives. Les demandes sont limitées à 10 par minute et par adresse source. Les comptes alice et loadbot gardent une authentification par mot de passe réservée aux scripts.

La double authentification fonctionne mais dépend du service de courriel : en cas d'indisponibilité, l'API renvoie 503 et détruit le défi. Le facteur e-mail est moins robuste qu'une application d'authentification ou une clé matérielle.

9.2 Gestion des secrets

Les valeurs sont chiffrées avec SOPS et age. La clé privée sert uniquement au déchiffrement. Les fichiers runtime sont protégés par des permissions 0700 et montés comme secrets Docker ; aucune valeur n'est inscrite dans le code ou suivie par Git. Les scripts reçoivent un chemin de répertoire de secrets plutôt qu'un mot de passe en argument ou variable d'environnement. Des tests vérifient ces règles d'hygiène.

9.3 Durcissement et exposition

Les cinq services Python tournent sans privilèges root, avec racine en lecture seule, capacités retirées, no-new-privileges et mémoire limitée. Aucun service ne monte docker.sock. Certaines images tierces nécessitent un système inscriptible ; Alloy tourne en root, en lecture seule, pour lire les journaux.

> Conséquence du durcissement. La racine en lecture seule du remédiateur oblige Ansible à placer ses fichiers temporaires sous /tmp. Sans les variables de redirection, les modules Ansible échouent et l'incident finirait en manual_review_required. Un syntax-check seul ne détecte pas le problème ; la configuration est couverte par tests/test_configs.py.

9.4 Où se trouve réellement la limitation de débit

Il n'y a aucune limitation de débit au WAF ni au reverse proxy. Les limites sont applicatives : 30 connexions par minute et par adresse côté authentification ; quatre emplacements de capacité partagés côté paiement ; et, après remédiation, un devis par identité et par seconde. Le WAF ne ralentit donc pas un flot de requêtes valides ; le contrôle d'admission applicatif agit après la détection.

9.5 Risques, traitement et risques résiduels

L'analyse EBIOS RM du démonstrateur est résumée en annexe E. Cinq risques sur sept sont ramenés à un niveau résiduel faible par des mesures documentées : détection et admission ciblée, transaction et idempotence, signature et action fixe, SOPS/age, corpus fermé sans outil d'action. Deux risques restent non résolus : perte du PC Hôte sans sauvegarde externe chiffrée, et indisponibilité des journaux ou du ML. La matrice est pédagogique ; l'acceptation formelle des risques par une direction et des responsables désignés n'a pas été réalisée.

9.6 NIS2 et DORA : comment ces cadres ont orienté les décisions

Le prototype n'est pas conforme à NIS2 ou DORA et ne revendique pas cette conformité. L'entreprise est fictive et aucune qualification réglementaire réelle n'a été établie. Ces textes ont servi de grille de lecture : le démonstrateur documente les incidents, définit des objectifs RTO/RPO et inventorie images, modèle et dépendances. Les écarts industriels sont résumés en annexe E. Aucun signalement réglementaire réel n'est démontré.

Références consultées le 19/10/2026 : directive (UE) 2022/2555, art. 4, 20, 21, 23 ; règlement (UE) 2022/2554, applicable depuis le 17 janvier 2025 ; méthode EBIOS RM (ANSSI).

10. Résultats des essais

Essais conduits sur le laboratoire réel, l'attaque lancée à la main depuis le PC Attaquant. **Deux séries de codes HTTP, à ne jamais mélanger** : l'observateur sonde `POST /api/payments` toutes les 0,5 s (impact métier), tandis que le client de charge compte ses propres requêtes `POST /api/quotes` (effet de l'attaque).

10.1 Cycle protégé de référence — 22 octobre 2026, 09:05 UTC

Quatre sources indépendantes décrivent le même cycle, et leur concordance a été vérifiée plutôt que supposée : la fenêtre M7 (09:04:40 → 09:06:33) couvre l'attaque et l'incident, son `first_alert_s` de 48,09 s tombe sur l'étape `containing` à 09:05:28,57, et le récapitulatif de recette désigne nommément la preuve d'observation qu'il a produite.

Le récapitulatif `recette-run-1792659908-c3ab69b0.json` rend un cycle de 86,0 s en mode `protected`, préparation 3/3 et **`acceptance`, `check_mardi` et `check_mercredi` tous les trois en `PASS`**, avec `attack_launched_by_runner: false`. La preuve `acceptance-1792659988.json` consigne **101 paiements fictifs** sondés — 91 `200` et 10 `503` —, un incident `recovered` en quatre étapes horodatées, la transaction témoin `695ef145` validée et le registre à **200 000 centimes** : détection → témoin en **6,88 s**. La preuve `m7-proof-20261022T090634Z.json` valide ses **4 contrôles sur 4** : pic *mesuré* à 6,25 req/s et occupation 4,0 contre une enveloppe normale de 1,37 et 1,0, **130,4 rejets de capacité**, 9 observations anormales sur 53, une alerte livrée, écart d'horloge Hôte – conteneur de 0,8 s.

Enfin le récapitulatif du PC Attaquant (run `92119f71`) compte **521 requêtes** `POST /api/quotes` en 30,4 s sans aucune erreur réseau : 84 `200`, 116 `503` et **321** `429`. Ce dernier code n'existe qu'une fois l'admission ciblée active : il prouve que le confinement a mordu *pendant* que la charge continuait. **Ce cycle établit une boucle complète sans intervention humaine, avec une reprise prouvée par une transaction métier** — pas par l'état d'un conteneur. Il n'établit ni un rappel de détection, ni une résistance à une attaque distribuée : la charge vient d'un poste unique et d'une seule identité.

10.2 Agrégat des essais de reprise protégée

Preuve	Date UTC	503 / sondes	Détect.→ témoin	Conclusion réellement permise
`acceptance-1792659988`	22/10 09:05	10 / 101	**6,88 s**	**Cycle de référence ci-dessus.** Le plus lent des 22 : l'étape `verifying` y a pris 6,4 s contre 2,3 à 2,6 s les 21/10.
`acceptance-1792585122`	21/10 12:17	16 / 115	2,47 s	Le plus rapide des 22 ; même chaîne, même témoin validé.
Agrégat des 22 reprises	19→22/10	10 à 19	**2,47** **2,83** **6,88 s**	**Reprise autonome reproductible : 22 cycles sur 22 sous la cible de 60 s** (minimum, médiane, maximum).
`recette-run-1792575980`	21/10 09:46	0 / 99	—	FAIL Aucun incident : l'attaque n'a pas saturé. **Ne prouve rien.**
`recette-run-1792576538`	21/10 09:55	0 / 98	—	NO_INCIDENT Observation calme menée à son terme ; reprise non testée.

Périmètre du calcul, et pourquoi deux décomptes coexistent. L'agrégat porte sur les **22 fichiers `acceptance-*.json`** — un par cycle d'observation, sans doublon — où *toutes* les conditions sont réunies : incident `recovered`, témoin `validated`, total exactement 200 000 centimes, quatre étapes présentes (`complete: true`). Les 39 preuves du dépôt se répartissent exactement en **22 reprises complètes + 14 observations sans aucun incident + 3 liées aux essais témoins** ; aucune n'est laissée de côté. Un *autre* décompte apparaît ailleurs dans ce dossier : les **14 récapitulatifs `recette-run-*.json`** lisibles. Il est plus petit parce qu'il ne couvre que les cycles pilotés par le lanceur hôte, écrit le 21/10 : les 13 reprises des 19 et 20/10 proviennent de `acceptance.py` lancé directement, sans récapitulatif. Les 10 cycles hôtes en mode protégé se répartissent en **8 `PASS`, 1 `FAIL`, 1 `NO_INCIDENT`**.

10 (suite) — Essais témoins et contrôles

10.3 Essais témoins sans remédiation — mesure d'impact uniquement

Récapitulatif	503 / sondes	Statut	Conclusion réellement permise
recette-run-1792653302 22/10 · 07:15	54 / 121	CONTROL_IMPACT_ OBSERVED	Deux incidents observés, sans remédiation. 45 % des paiements fictifs refusés : impact constaté, pas une reprise.
recette-run-1792590023 21/10 · 13:40	48 / 121	CONTROL_IMPACT_ OBSERVED	Essai répété : 40 % refusés ; check_mardi passe et la charge est mesurée.
recette-run-1792589548 21/10 · 13:32	54 / 122	CONTROL_NOT_ INHIBITED	Essai invalide conservé comme tel : un incident a été recovered malgré le mode témoin.
recette-run-1792589278 21/10 · 13:27	0 / 113	CONTROL_NO_ IMPACT	Aucun rejet de capacité ni incident ; aucun impact établi.

Lecture du troisième essai. acceptance.py a rendu passed: true parce qu'un incident recovered existait. Ce verdict ne prouve pas un essai témoin valide. Le statut CONTROL_NOT_INHIBITED signale que la remédiation est intervenue malgré l'inhibition.

10.4 Écart entre les deux modes

Grandeur	Mode protégé	Mode témoin	Lecture
Paiements fictifs refusés	10–19 / ~101–115 (10–17 %)	48 et 54 / ~121 (40–45 %)	Sous charge comparable, l'admission ciblée divise par environ quatre la proportion de refus.
État final	recovered	observed	États distincts ; aucune reprise n'est revendiquée en mode témoin.
Transaction témoin	validée (22/22)	absente	La reprise n'est prouvée qu'en mode protégé.
Total du registre	200 000 c.	200 000 c.	La disponibilité baisse sous charge ; l'invariant métier reste tenu.

10.5 Vérifications automatisées — état vérifié pour ce dossier

354 tests Python sont collectés dans l'état actuel du dépôt. La dernière exécution complète consignée le 22/10 a réussi 352 tests et en a échoué 2 : le test de chaîne native, car le playbook Ansible ne s'exécute pas dans l'environnement de rédaction, et la conformité OpenAPI, car deux routes d'authentification présentes dans le code ne figurent pas encore dans le contrat. Ces échecs ne sont pas présentés comme des succès.

Côté interface, 55 tests sur 55 réussissent. Le build Vite exécuté pour cette mise à jour aboutit (241,48 kB de JavaScript ; 22,08 kB de CSS).

Cette revue a reconfirmé les 354 tests par une collecte, mais n'a pas achevé une nouvelle exécution complète de la suite Python. La trace pytest.txt du 19/10 est historique et ne décrit pas cet état. La preuve de référence enregistre c11ec6f avec dirty: true ; le dépôt est aujourd'hui au commit 4a1430d. Leur équivalence exacte n'est pas établie.

11. PCA/PRA, limites et perspectives

11.1 Écart aux objectifs

Objectif	Cible	Mesuré	Écart / portée
Détection → service rétabli	< 60 s	2,47 à 6,88 s médiane 2,83 s	Tenu sur 22 cycles ; tous sous la cible.
Début de charge → détection → transaction témoin	< 10 s < 45 s	~5,5 s ~12,3 s	Mesuré sur un seul cycle horodaté ; à répéter.
RTO : première indisponibilité → transaction confirmée	< 45 s	Non mesuré formellement	L'écart observé entre premier paiement refusé et premier servi ne mesure pas la durée totale de dégradation.
RPO sur ce scénario	0 transaction confirmée perdue	Total à 200 000 c. sur 22 cycles	Tenu pour les essais ; restauration après perte de l'Hôte non testée.

11.2 Dépendances et points de défaillance

Le laboratoire repose sur un PC Hôte et une base uniques, sans haute disponibilité physique ni TLS applicatif. Le paiement utilise un seul worker car le compteur de capacité est en mémoire. Le proxy partage son budget de connexion : sous saturation, l'accès opérateur peut également être gêné. La sauvegarde reste sur l'Hôte ; sa perte n'est pas couverte par une copie externe chiffrée. Le prototype ne prétend pas être une plateforme de paiement de production.

11.3 Limites et perspectives

Sur 156 observations de trafic normal indépendant, aucune fausse série n'a été produite ; cela ne prouve pas l'absence générale de faux positifs. Sur 39 preuves, 14 ne montrent aucun incident parce que la charge n'a pas saturé le service : ce n'est pas un taux de non-détection. Le rappel reste NOT MEASURED.

Pour un contexte industriel, les pistes retenues sont de mesurer le rappel sur davantage de cycles, de séparer les ressources devis/paiement pour supprimer leur compétition, et de tester une restauration depuis une sauvegarde chiffrée hors de l'Hôte.

12. Contribution des six membres

Répartition des responsabilités retenue par l'équipe. Git ne permet pas d'établir les contributions individuelles.

Membre	Responsabilité retenue
Sofiane CHELGHOUM	Chef de projet, coordination et gouvernance.
Yanis KREIR	Développement de l'API paiement et du registre.
Walid AIT DAOUD	Infrastructure, Docker Compose et Ansible.
Gabriel ROBERT	IA/Data et détection ML.
Hafsa BELKASMI	Cybersécurité, authentification, secrets et WAF.
Hafsa KODAD	Interface, observabilité et assistant documentaire.

Annexe A — Commandes de déploiement et d'essai

Prérequis : Docker Desktop avec WSL2, SOPS et une clé *age*. Aucune de ces commandes ne contient de secret ; aucune ne supprime de volume.

Étape	Commande
Déchiffrer et installer les secrets	`python scripts/init_secrets.py`
Contrôler la configuration	`docker compose config --quiet`
Déployer la pile	`python -m ansible.cli.playbook -i localhost, infra/ansible/deploy.yml`
Télécharger le modèle local	`docker compose exec ollama ollama pull qwen2.5:1.5b`
Jalon « pile levée », lecture seule	`python scripts/check_lundi.py`
Trafic normal pour la baseline	`python scripts/normal_traffic.py --seconds 240`
Collecter la baseline	`docker compose exec detector python -m services.detector.collect_baseline`
Entraîner le modèle	`docker compose exec detector python -m services.detector.train`
Recette protégée, de bout en bout	`python scripts/run_recette.py`
Essai témoin sans remédiation	`python scripts/run_recette.py --mode control`
Preuve du jalon de détection	`python scripts/check_mardi.py --last 120`
Preuve de la boucle complète	`python scripts/check_mercredi.py --max-seconds 60`
Charge, sur le PC Attaquant	`python attack_lab.py --lab-authorized --seconds 30`

> **Ne jamais utiliser** `docker compose down -v` pour rétablir la démonstration : cela efface le registre. Le reset documenté entre deux essais supprime *uniquement* le fichier de politique de confinement, puis redémarre le service de paiement. Les incidents historiques sont conservés ; l'observateur distingue les nouveaux.

Annexe B — Extrait de configuration : routes publiées

Extrait non sensible de `infra/nginx/default.conf`. Toute route non énumérée renvoie `404`.

Route publique	Service interne	Accès
`/auth/token`, `/auth/operator/challenge`, `/auth/operator/verify`	authentification	public
`/api/quotes`, `/api/payments`, `/api/accounts`, `/api/ready`	paiement	session requise sauf `/api/ready`
`/ops/status`	remédiateur	rôle opérateur, **GET seulement**
`/ops/control-trial`	remédiateur	rôle opérateur, GET et POST
`/ops/ml`	détecteur	rôle opérateur, via sous-requête d'autorisation
`/rag/ask`	assistant	rôle opérateur

Il n'existe **aucune** route publique vers `/control/contain` (service de paiement) ni vers `/alerts` (remédiateur) : ce sont les deux seuls points de contrôle, et ils restent sur des réseaux internes, protégés l'un par un secret dédié, l'autre par une signature HMAC.

Cloisonnement réseau (§3.2)

Réseau	Isolé	Services attachés	Ce que le cloisonnement empêche
`front`	non	waf, edge, remédiateur	Seul `waf` y publie un port.
`app`	**oui**	auth, paiement, remédiateur, frontend, edge, rag	Le trafic applicatif ne sort jamais de l'Hôte.
`data`	**oui**	base, paiement	La base n'est joignable que par le service de paiement.
`control`	**oui**	paiement, remédiateur	L'action de confinement n'est atteignable que depuis le remédiateur.
`observe`	**oui**	paiement, remédiateur, détecteur, edge, Prometheus, Loki, Alloy, Grafana	Métriques et journaux restent internes.
`ai`, `mail`, `mgmt`	non	ollama + rag ; auth ; Grafana	IA locale et envoi du code de connexion séparés du reste ; Grafana sur `127.0.0.1`.

Bornes du client de charge (§5.2)

Borne	Valeur	Où elle est appliquée
Débit	20 req/s au maximum (une toutes les 50 ms)	boucle d'envoi
Concurrence	12 requêtes en vol, 12 connexions	boucle d'envoi, `httpx.Limits`
Volume, durée	700 requêtes ; `--seconds` de 5 à 35, **30 par défaut**	boucle d'envoi, contrôle des arguments
Cible	`POST /api/quotes`, corps `{"amount_cents": 100}`	code du script
Périmètre, accord	IP littérale de `192.168.137.0/24` ou loopback ; `--lab-authorized` obligatoire	`resolve_target`, contrôle des arguments

Séries Prometheus (§8.1)

Série	Type	Signification
`lab_requests_total{route,status}`	compteur	Réponses métier par route et par code. Les séries sont pré-touchées au démarrage : sans cela, après un redémarrage, la requête du détecteur ne renvoie rien et il se déclare en erreur de télémétrie.
`lab_capacity_denied_total`	compteur	Requêtes rejetées parce que la capacité est épuisée. Signal direct de l'incident.
`lab_active_work`	jauge	Emplacements de capacité aval simulée occupés (0 à 4).
`lab_contained`	jauge	1 quand l'admission ciblée est active. Permet de distinguer « avant » et « après » confinement sur un graphique.
`lab_duration_seconds{route}`	histogramme	Temps de traitement par route.

Annexe C — Index des preuves citées

Fichier	Rôle	Ce qu'il établit
recette-run-1792659908-c3ab69b0.json acceptance-1792659988.json m7-proof-20261022T090634Z.json	Cycle protégé de référence 22/10 · 09:05	PASS sur les trois contrôles ; reprise en 6,88 s, transaction témoin validée, registre à 200 000 centimes ; M7 4/4 avec 130,4 rejets de capacité.
Récapitulatif PC Attaquant run 92119f71 · non versionné	Charge	521 POST /api/quotes en 30,4 s : 84 réponses 200, 116 réponses 503 et 321 réponses 429. Donne l'heure exacte de lancement.
22 acceptance-*.json avec témoin validé	Agrégat	Détection → témoin : 2,47 / 2,83 / 6,88 s ; 22/22 sous 60 s.
recette-run-1792653302 / 1792590023	Essais témoins	54/121 et 48/121 paiements refusés ; incidents observed, sans remédiation.
recette-run-1792589548-4afbab60.json	Essai invalide	Remédiation intervenue malgré l'inhibition ; résultat conservé comme échec.
calibration-20261020T065603Z.json	Calibration	nu=0,02, consecutive=3 ; 0 fausse série sur 156 observations normales.
ml-report.json ; ml-baseline.json	Modèle	Baseline de 90 observations (72/18), empreinte du jeu ; attack_recall: NOT MEASURED.
18 m7-proof-*.json validées sur 23	Détection	Pic 4,88–6,63 req/s contre une enveloppe à 1,37 ; de 7 à 19 observations anormales.

Qualité du dossier de preuves. Parmi 50 fichiers recette-run-*, 35 durent moins de 10 ms : ils viennent des tests automatisés et ne sont pas comptés comme essais réels. Un récapitulatif est illisible (recette-run-1792567578-d857f112.json) ; l'acceptance correspondante est lisible et comptée, mais pas ce résumé. pytest.txt et frontend-build.txt datent du 19/10 et sont historiques.

Traçabilité : les preuves du cycle de référence enregistrent c11ec6f et dirty: true. Le dépôt actuel est au commit 4a1430d. L'équivalence du code exécuté avec ce commit n'est pas établie ; les résultats sont rattachés aux fichiers de preuve horodatés.

Annexe D — Documents de référence du dépôt

docs/01-EXIGENCES.md · 02-ARCHITECTURE.md · 03-INTERFACES.md et docs/openapi/*.json · 04-ATTAQUE.md · 05-ML-RAG.md · 06-SECURITE.md · 07-GOUVERNANCE.md · 08-PCA-PRA.md · 09-RECETTE-RENDU.md · INSTALL.md · VALIDATION.md. Ces documents décrivent les exigences, l'architecture, les contrats, le scénario, le ML/RAG, la sécurité, la gouvernance, la continuité et les validations.

Annexe E — Matrices de gouvernance

Tableaux détaillés des §9.5 et §9.6. Échelle qualitative 1 (faible) à 4 (critique) ; le produit V×G hiérarchise en interne et n'est pas un score EBIOS officiel. Les risques signalés comme non résolus le restent.

Risque	V×G initial	Traitement et preuve	Résiduel
Indisponibilité devis/paiement	4×3	Détection ML, admission ciblée, transaction de vérification.	2×3
Double débit lors d'un rejeu	3×4	Transaction base, verrou ordonné, idempotence ; test concurrent.	1×4
Abus de remédiation	3×4	HMAC, fraîcheur, action fixe, aucun socket Docker.	1×4
Secrets dans les livrables	3×4	SOPS/age, exclusion de l'archive et revue du manifeste.	1×4
Instruction malveillante via le RAG	3×3	Corpus contrôlé, aucun outil d'action ni secret de contrôle.	2×2
Perte du PC Hôte	2×4	Sauvegarde avec empreinte ; copie externe chiffrée à organiser.	2×4
Journaux ou ML indisponibles	2×3	État d'erreur explicite ; aucune fausse reprise.	2×3

NIS2 / DORA — thèmes, apports et écarts

Thème	Ce que le prototype apporte	Écart industriel
Gouvernance du risque TIC	Analyse EBIOS synthétique, décisions tracées et responsabilités d'équipe définies.	Validation de direction, politique formelle, contrôles indépendants.
Gestion d'incident	Détection, chronologie horodatée, action autorisée, états recovered/failed distincts.	Qualification réglementaire et procédure de notification.
Continuité et restauration	Objectifs RTO/RPO écrits, sauvegarde et contrôle d'empreinte.	Sites indépendants, exercices réguliers, restauration hors machine testée.
Tests de résilience	Scénario de laboratoire borné, résultats automatisés documentés et preuves conservées.	Programme adapté aux obligations applicables.
Accès et secrets	Sessions signées, rôles, SOPS/age, double authentification opérateur.	IAM complet, rotation, revue des droits et TLS.
Prestataires TIC	Inventaire des images, du modèle et des dépendances.	Diligence, contrats, registre réglementaire et stratégie de sortie.
Signalement	Journal d'incident pédagogique.	Aucun signalement réel ; rôles, seuils et délais à valider.

La machine Hôte peut altérer ses propres journaux. Cette architecture de laboratoire ne démontre pas l'inaltérabilité industrielle des preuves. NIS2 et DORA servent de repères de gouvernance ; le prototype ne revendique pas leur conformité.